ME303 - STUDY ORIENTED PROGRAM

BITS PILANI

DEPARTMENT OF CSIS

Value-Aware Adaptive Multi-Robot Exploration under Resource Constraints

Semester Progress Report

Author:
Chaitanya Goyal (ID: 2024A7PS0557P)

Supervisor:
Prof. Avinash Gautam

Date: November 2026

1 The Short Version

This report is a check-in: what I have built so far, what I found, and where the project is going next.

In one paragraph: when a team of robots explores an unknown building, the first few minutes reveal a lot of new space, but the last few minutes are mostly long drives to find small leftover corners. Every minute still costs battery, messages between robots and planning time. So at some point, exploring more is not worth it. I built a small simulator to study this, taught a program to decide *when* the team should stop, and tested whether that decision is actually worth learning. Following the review, the bigger goal is now to decide *how much exploration effort* (how many robots, how much messaging) is still worth spending, with stopping being the case where zero robots keep working.

What I have so far:

- **A working simulator** of robots exploring random buildings, with costs for driving, messages and planning.
- **A learned stop rule** that looks at the team's situation after every round and decides whether one more round is worth it.
- **Stage A done:** the problem is real. If robots explore until the map is complete, about a third of all their driving happens after the best moment to stop. But this is only true when driving and messaging are expensive.
- **Stage B done:** the reinforcement-learning way of training is clearly better than the obvious "learn from the right answers" way. A big model is not needed; even a simple one works.
- **A fair comparison** against five common stopping rules. In normal cases my rule ties the best one; in bad cases it loses far less.
- **All code and results** on GitHub, so everything can be re-run.

2 The Problem, in Plain Words

2.1 How robots explore a building

Imagine the building as a big sheet of graph paper. Each square is either free space, a wall, or unknown. At the start, every square is unknown. Each robot can only see the squares near it (in my simulator, within 4 squares, and walls block the view).

A **frontier** is the edge between what we know and what we don't: a known free square right next to an unknown one [18]. If a robot drives to a frontier, it gets to see new space. So exploration is basically: find frontiers, send robots to them, repeat. When there are several robots, each one should go to a different frontier so they don't waste effort on the same area [4].

I group the robots' work into **cycles**. One cycle is 6 robot steps: look around, share maps, pick frontiers, move. After every cycle, the team has a chance to decide: keep going, or stop?

2.2 Why stopping is a real question

The classic answer is “stop when there are no frontiers left”, meaning the whole building is mapped. But look at what happens in practice (Figure 1). Early cycles reveal whole rooms. Later cycles reveal tiny pockets that need long drives. The cost keeps going up at roughly the same speed the whole time, but the new information slows down a lot.

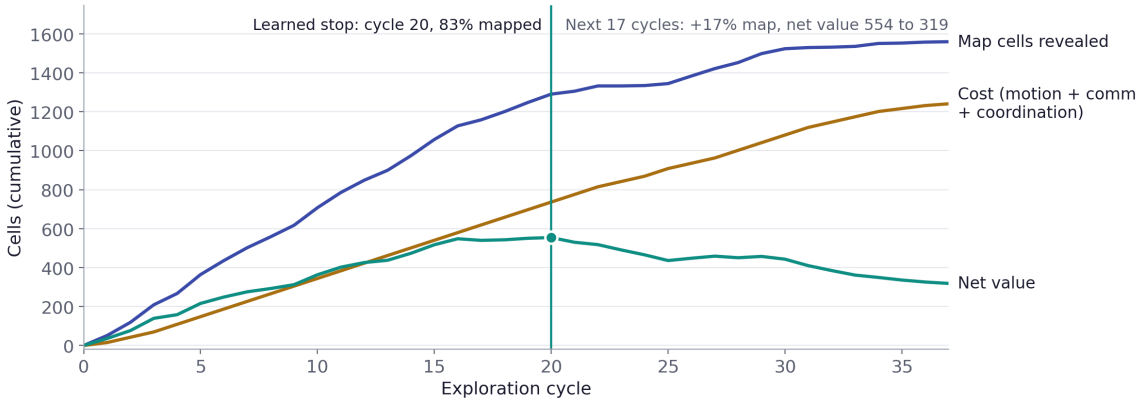


Figure 1: One test run with 3 robots on a 40×40 map. Blue: total map found. Orange: total cost. Green: net value (map found minus cost). Net value peaks around cycle 20, when 83% of the free space is mapped. The last 17 cycles add 17% more map but drop the net value from 554 to 319.

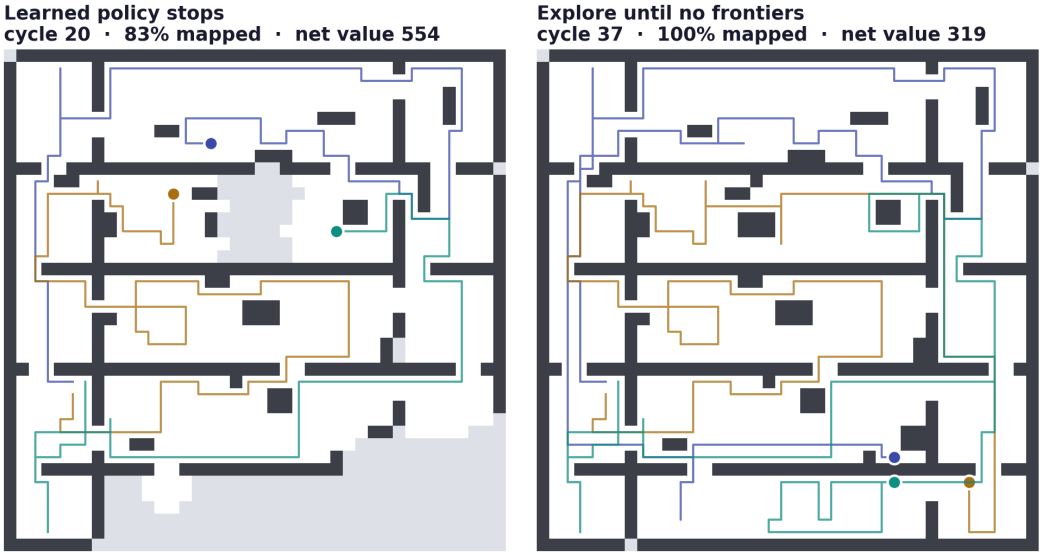


Figure 2: The same map. Left: where the learned rule stopped (grey areas are still unknown). Right: exploring until nothing is left. The extra map on the right was expensive to get.

So the real question is not “*can* the team explore more?” but “*is* one more cycle worth what it costs?”

2.3 How I put a number on it

To compare "map found" and "cost" I need them in the same units. I measure everything in map squares. The score of one cycle is:

$$\text{score of a cycle} \;=\; \text{new squares seen} \;-\; c_{\text{move}} \cdot (\text{robot steps}) \;-\; c_{\text{comm}} \cdot N(N-1) \;-\; c_{\text{coord}}$$

In words: what you found, minus what driving cost, minus what the messages cost (every one of the N robots sends its map to every other robot, which is $N(N-1)$ messages), minus a fixed planning cost. By default I use $c_{\text{move}} = 2$, $c_{\text{coord}} = 2$, and a message cost of 0.05, 0.2 or 0.6 for cheap, medium and expensive messaging.

If you stop after T cycles, your total is just the scores of those T cycles added up. After a run is over, I can look back and find the cycle where this total was highest. I call that the **best stop in hindsight**. It needs knowledge of the future, so no robot could actually use it; it is only the 100% mark I grade everything against.

3 What Changed After the Review

The review raised two big points, and both changed the project.

1. The original claim was too strong. The first proposal said, roughly, that nobody had learned when a team of robots should stop exploring. The review pointed out papers that already deal with stopping exploration: Amorín et al. [2] stop a whole robot team based on how much new information it is still expected to gain; Placed and Castellanos [13] stop a single robot when its uncertainty stops improving; Luperto et al. [11] train a neural network to judge whether a map looks complete; PRoID [9] lets each robot decide when to go back and share its data; WiSER-X [8] lets each robot finish on its own schedule. None of them does exactly what I do, but together they mean "stopping has not been studied" is wrong.

So the claim is now narrower: *to my knowledge, no earlier work **learns**, for the **whole team**, how much exploration effort is still worth spending, counting the costs of driving, messaging and coordination.*

2. A yes/no stop does not justify a fancy two-level learning system. If the only decision is "continue or stop", a professor could fairly ask: why not just use a simple threshold or a regression model? The review's answer is to make the top-level decision richer: instead of yes/no, decide **how many robots stay active**. For example, $8 \to 5 \to 2 \to 1 \to 0$. Stopping is then just the moment the number hits zero. This turns the project into a question about *how much exploration effort is worth buying*, and it gives a real reason for two levels: the top level decides how much effort to spend, the bottom level decides how the remaining robots use it.

The review also asked for a step-by-step plan before any heavy machine learning (Stages A to E, Section 7). I followed that order this semester.

4 What I Built

Everything below is in the GitHub repository (`dev-goyal-chaitanya/marl-simulations`) and runs on a normal laptop.

4.1 The simulator

- **Maps:** random indoor buildings on a grid: rooms, doors, and furniture blocks. Sizes 32×32, 48×48 and 64×64.
- **Robots:** 2 to 8 robots, all starting together. Each sees squares within 4 cells in line of sight.
- **Movement:** each robot drives to the nearest frontier, and robots are spread out so they don't chase the same one. This is a simple fixed rule. It is not what I am trying to improve; it is just the "engine" the robots run on.

4.2 The learned stop rule

The idea is simple. After each cycle, the team asks: *if we keep going, how much net value do we expect to gain from here on?* Call that number Q. If Q is zero or negative, stop.

Of course the team can't know the future, so a program has to *estimate* Q from what the team can see right now. I describe the team's situation with 7 simple numbers:

1. how much of the map is already known,
2. unknown squares near the planned targets, divided by the cost of one cycle,
3. how well the last cycle paid off (gain divided by cost),
4. how well the cycle before that paid off,
5. how far away the planned frontiers are,
6. what share of the robots still have somewhere to go,
7. how much frontier is left, relative to the map size.

All of them are fractions or ratios, not raw counts. That is on purpose: a ratio like "gain per unit cost" means the same thing for a small map and a big map, for 2 robots and for 8, so one rule can be used everywhere without retraining.

4.3 How the program learns

This is where reinforcement learning (RL) comes in. Put simply, RL means learning what to do from scores rather than from an answer key.

I used a method called **fitted Q-iteration** [6]. Here is the whole idea:

1. Run 300 practice maps (2 robots, 32×32) all the way to the end, and record the team's 7 numbers and the score of every cycle.
2. Score each recorded run under 4 random sets of prices for driving, messages and planning. That gives 1,200 runs from the same 300, and stops the program from memorising one price.
3. Start with a rough guess of Q. For every recorded moment, build a target: *this cycle's score, plus the guessed value of the next moment (or 0 if it's better to stop there)*. Fit a model to those targets. That model is the new guess.

4. Repeat step 3 twenty-five times. Each round, the guesses get better.

The model in step 3 is an “extra-trees regressor”: a group of 80 decision trees whose answers are averaged. It’s fast and works well with small data.

The “stop when the value of continuing is zero or less” idea is old: it comes from finance, where people use it to decide when to cash in an option [10, 17]. I am applying it to robot exploration.

4.4 What I compare against

A result means nothing without something to compare to. These are the rules I compare with:

- **Explore until done:** stop only when no frontiers are left [18, 4].
- **Fixed budget:** always stop after a set number of cycles (tuned to 19).
- **Coverage threshold:** stop once a set fraction of the grid is known (tuned to 84%).
- **Myopic rule:** stop right after the first cycle that lost value.
- **Gain-vs-cost rule:** a hand-made rule that stops when the expected gain falls below the cycle cost divided by 1.6.
- **Best stop in hindsight:** the 100% mark, not a real rule.

“Tuned” means I picked the number that worked best on the training maps, which gives these rules a fair chance.

4.5 How I judge results

- **% of best:** total value a rule got, divided by the total value of the best stop in hindsight.
- **Value lost:** how much value a rule missed compared with the best stop, as a percentage of the map. This is more stable than “% of best” when the best value is small.
- **Worst case:** the setting where a rule did worst. A rule that is fine on average but terrible somewhere is risky on a real robot.
- **Generalisation:** train once on 2 robots and small maps, then test on 36 different settings (2, 4, 6 or 8 robots; three map sizes; cheap, medium or expensive messaging; 40 maps each) *without retraining*.

The way I report averages, worst cases and confidence ranges follows the advice in Agarwal et al. [1], which is about how to compare learning methods honestly.

5 Results So Far

5.1 Stage A: is the problem even real?

Before building anything clever, the review asked a simple question: *how much value do we actually lose by exploring until the map is done?* If the answer is "almost nothing", the whole project is pointless.

I took 1,440 finished runs (the 36 settings above, 40 maps each), found the best stop in hindsight for each, and measured what the extra exploration cost.

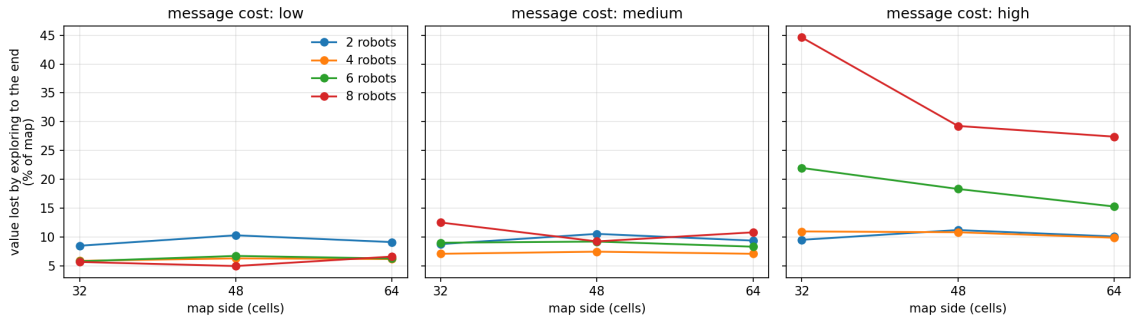


Figure 3: Value lost by exploring until done, as a percentage of the map. Each panel is one message cost; each line is a team size.

Team size	Cheap messages	Medium	Expensive
2 robots	9.3	9.6	10.3
4 robots	6.1	7.2	10.6
6 robots	6.3	8.8	18.5
8 robots	5.8	10.8	33.8

Table 1: Value lost by exploring until done (% of map), averaged over the three map sizes.

What this shows:

- On average, exploring until done throws away **11.4% of the map's value**, and **36% of all driving** happens after the best stopping point.
- It gets worse with **bigger teams** and **expensive messaging**. With 8 robots and expensive messages, a third of the value is lost.

The honest catch. All of this depends on how much I charge for driving. I re-scored the same runs with different driving prices (medium message cost):

Cost per robot step	Value lost (% of map)	Driving after the best stop
0.5	1.0	4.8%
1	2.2	9.6%
2 (default)	9.1	33%
3	27.3	68%

Table 2: How the size of the problem depends on the price of driving.

If driving is cheap, just explore everything; there is nothing to fix. The project matters

when driving or messaging is expensive: big teams, small batteries, costly radio. So one of the next steps is to use realistic costs (from real robots), or to show results across the whole range.

5.2 Stage B: do we actually need reinforcement learning?

The review asked whether a simpler method would do. So I tried two ways of making the training targets, each with four kinds of model:

- **RL-style (fitted Q-iteration):** the target uses the program's own estimate of what happens next, as described in Section 4.3.
- **Hindsight answers:** plain supervised learning. After each run, label every moment with the best value that was still possible from there, and train the model to predict that label. This looks like the obvious approach.

Training target	Model	Avg value lost	Worst setting	Settings <50%
RL-style	**extra trees**	**3.6**	**9.9**	**1**
RL-style	random forest	3.7	10.3	1
RL-style	linear	3.5	13.0	4
RL-style	neural net	4.8	12.1	1
Hindsight	linear	6.7	30.4	6
Hindsight	neural net	5.4	37.4	5
Hindsight	random forest	9.0	35.7	8
Hindsight	extra trees	9.6	39.0	8
Coverage threshold (tuned), for reference		4.7	25.3	5

Table 3: Value lost (% of map) over the 36 test settings, no retraining. Lower is better.

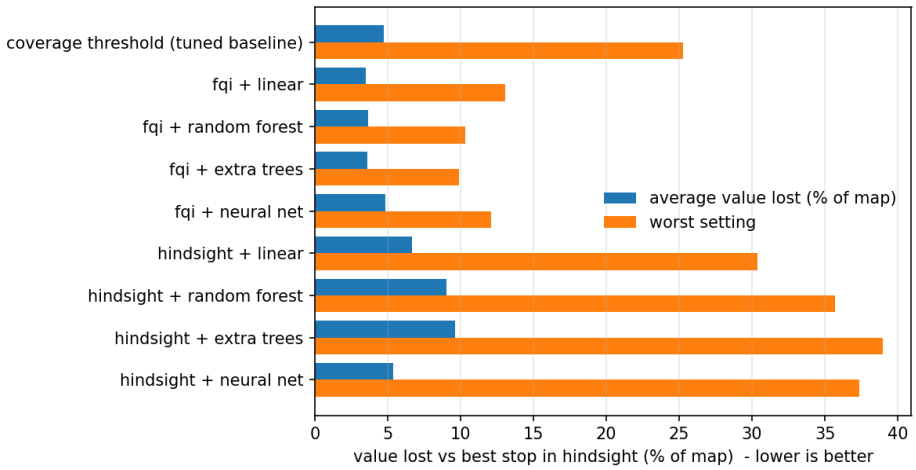


Figure 4: The same comparison as a chart: average value lost (blue) and the worst setting (orange).

Two clear lessons:

1. **How you build the target matters much more than which model you use.** Every RL-style version beats every hindsight version, especially in the worst case (10–13% lost versus 30–39%).

2. **A big model is not needed.** Even a straight-line (linear) model does well with the RL-style target. A neural network adds nothing here.

Why do hindsight answers do worse? The label says "this much value was still possible", but that assumes the team will later stop at exactly the right moment, which it can't, because it doesn't know the future. So the labels are too optimistic, the program thinks continuing is worth more than it is, and it stops too late. People who study stopping problems know this trap; using future knowledge in the targets makes estimates too high [10]. The RL-style target only promises what the rule itself can actually deliver.

The learned estimate also tracks reality well: on test maps, its estimate of "value still left" has a correlation of 0.88 with the true value, and it gets the basic question "is it still worth continuing?" right 89% of the time.

5.3 Stage C (so far): how does it compare with the usual rules?

Stop rule	% of best	Avg value lost	Worst setting
Learned value-aware stop	**91**	**3.6**	**9.9**
Coverage threshold (tuned)	92	4.7	25.3
Gain-vs-cost rule (tuned)	89	6.9	15.8
Fixed budget (tuned)	91	9.1	37.1
Myopic (stop after a losing cycle)	68	11.4	24.5
Explore until done	74	11.4	44.6

Table 4: "% of best" is measured on 140 test maps of the training setting (2 robots, 32×32, medium messaging). The other two columns are value lost (% of map) over the 36 settings, without retraining.

In plain words:

- **In normal cases, it's a tie.** A well-tuned "stop at 84% mapped" rule does as well as my learned rule (92% vs 91%).
- **In bad cases, the learned rule is much safer.** Its worst setting loses 9.9% of the map, against 25.3% for the coverage rule. That's because a fixed threshold that suits 2 robots doesn't suit 8, while the learned rule adapts without being re-tuned.

5.4 Honest weak spots

- With **2 robots on the larger maps**, the learned rule sometimes stops 20–33 cycles too early. I don't know why yet; my guess is that it meets situations it never saw in training.
- Only **one type of building** (rooms with furniture). Mazes, warehouses and cluttered spaces are not tested yet.
- Only **one training run**. I need to repeat with several random seeds to make sure the results are not luck.
- The **cost numbers are my choice**, not measured from real robots.

6 Where the Project Is Heading

6.1 From "when to stop" to "how much effort to spend"

Following the review, the top-level decision changes from a yes/no to a choice of effort level, mainly the number of robots that keep working (and later, how many messages to allow). The value of choosing an effort level a is, in words:

$$\text{value of } a \;=\; \text{expected extra map from } a \;-\; \text{expected cost of } a$$

where the cost covers driving, messages, planning and, later, the risk of a robot failing. The team picks the effort level with the highest value. Stopping (0 robots) is always worth exactly 0, so the team stops when no effort level beats 0. My current rule is just the simplest case of this, with only two choices: continue or stop.

6.2 Why this needs two levels

With team size in the picture, there are naturally two decisions at two speeds:

- **Top level (the "manager"):** once per cycle, decide how many robots keep working and how much they may talk. Zero robots means the mission ends.
- **Bottom level (each robot):** every step, decide where to go, given whoever is still active.

This is where multi-agent reinforcement learning (MARL) can come in later: robots that learn to explore together, trained with a method like MAPPO [19]. During training they can use the full map to learn faster, but on a real mission each robot only uses what it can see itself. The idea of a learned "when to finish" inside a two-level system comes from hierarchical RL [14, 7].

But, as the review says, MARL is a *tool*, not the contribution. It only gets added once the simpler stages show it is needed.

7 The Stage-by-Stage Plan

Stage	Question	Status
A	Is the problem real? How much value does exploring until done throw away?	Done
B	Can the remaining value be learned? Is RL needed, or is a simple model enough?	Done
C	With robot movement fixed, how do all the stopping rules compare, including rules based on the closest papers?	In progress
D	Can the team shrink as value drops ($8 \to 6 \to 3 \to 1 \to 0$) and keep almost the same map for much less cost?	Next
E	Two-level learning: a manager that picks team size plus robots that learn where to go (MAPPO).	Next semester

Table 5: The staged plan from the review, with where I am.

7.1 What I will do for the rest of this semester

When	Build	Output
Nov, weeks 1–2	Realistic cost values or a full sweep of costs; three new comparison rules based on Amorín [2], Luperto [11] and PRoID [9]	Full Stage C table
Nov, weeks 2–4	Team shrinking (Stage D)	Learned shrinking vs a fixed schedule vs the full team
Late Nov	New building types, 5 training seeds, look into the 2-robot failure	Generalisation results
Early Dec	Write-up of Stages A–D	Report and an outline of a first paper

Table 6: Proposed timeline (to be adjusted to the semester calendar).

Team shrinking (Stage D), in a bit more detail. Each cycle, the manager picks how many robots stay active. Retired robots stop moving and stop messaging, so they cost nothing. To train it, I will record runs where robots retire at random times, so the program sees every team size, and then use the same fitted Q-iteration with the number of active robots as an extra input. Success would mean: almost the same map value as the full team, at clearly lower driving and messaging cost.

Tests decided before training. The review asked for this explicitly, so the results can't be cherry-picked afterwards:

- new building types: maze, warehouse, cluttered;
- bigger maps: up to 128×128;
- bigger teams: train on 2–4 robots, test on 5, 6, 8 and 10;
- worse communication: limited range, lost messages, delays;
- a sweep of cost settings, to show the trade-off between map value and cost (the review calls this a Pareto analysis).

7.2 Questions I need answered

1. Which cost values are realistic? Should I take them from the robots in the MRTA lab setup, or report a sweep?
2. When a robot is retired, does it park where it is, or drive back to base (and pay for that trip)?
3. Is retiring permanent, or can a robot be brought back later?
4. Is a first paper on value estimation and stopping alone (no MARL) a good first target?

8 Papers I Have Been Working With

The shared literature sheet currently has 33 entries. The target is 50 by the end of November and 80 before the MARL stage starts, each recorded against the review's questions

(what decision it makes, learned or hand-made, single robot or team, which costs it counts, and so on). These are the papers that shaped the work so far.

8.1 Closest work on stopping exploration

- **Amorín et al. 2019** [2]: a stop rule for a whole robot team based on how much new information the team is still expected to gain. The closest earlier work; it becomes one of my comparison rules.
- **Placed and Castellanos 2022** [13]: one robot stops when its uncertainty about the map and its own position stops improving. A principled rule, but single-robot and it doesn't count driving or messaging costs.
- **Luperto et al. 2026** [11]: a neural network looks at an image of the partial map and says "explored" or "not explored". Learned, but from labelled examples, single-robot and not cost-aware. Their code is public, so I can compare against it.
- **PRoID (Kim et al. 2026)** [9]: in a team, each robot decides when to stop exploring and go back to share its data, using a formula. Decided per robot rather than for the team, and not learned.
- **WiSER-X (Jadhav et al. 2026)** [8]: robots explore using wireless signals instead of sharing maps, and each one can finish on its own. Relevant to the long-term goal of making the decision without a central controller.

8.2 Where my method comes from

- **Ernst et al. 2005** [6]: fitted Q-iteration with tree models. This is exactly my training method.
- **Longstaff and Schwartz 2001; Tsitsiklis and Van Roy 1999** [10, 17]: estimating "the value of continuing" from data and stopping when it isn't worth it. My stop rule is this idea moved from finance to robots. Becker et al. [3] do the same with neural networks.

8.3 Background on exploration and team learning

- **Yamauchi 1997** [18]: invented frontiers. My robots use them.
- **Burgard et al. 2005** [4]: sending each robot to a different frontier by weighing gain against travel. My robots' movement rule follows this.
- **Tan et al. 2023; MARVEL 2025** [16, 5]: recent work where robots *learn* where to go. Their missions end at a fixed time or coverage, which is the gap I'm working on.
- **MAPPO (Yu et al. 2022)** [19]: a simple, strong method for training robot teams. The planned method for Stage E.
- **Han et al. 2019; Sutton et al. 1999** [7, 14]: two-level learning where agents learn when to finish a sub-task. Close in spirit, but they end sub-tasks, not the whole mission.

- **Szostak and Cohen 2024** [15]: a team of agents that learns when to stop collecting samples. A learned team stop, but with no map and no driving.
- **Oliehoek and Amato 2016** [12]: the textbook way to describe a team where each member sees only part of the world. I use it to describe the problem formally.
- **Agarwal et al. 2021** [1]: how to report learning results fairly. My evaluation follows it.

9 What I Learned Along the Way

A few things that were not obvious to me at the start:

- **Check whether the problem is real before solving it.** Stage A showed the problem disappears when driving is cheap. That was humbling, and it changed what I need to do next.
- **The training target matters more than the model.** I expected a neural network to win. Instead, the choice of target decided everything, and a linear model was nearly as good.
- **Average results hide the important part.** The learned rule only ties the best simple rule on average. The real difference shows up in the worst cases.
- **Claims need care.** "Nobody has done this" was wrong. A narrower, precise claim is easier to defend and more honest.

10 Summary

This semester I built the simulator, the cost model and a learned stop rule, and completed the first two stages of the plan from the review. The problem is real when driving and messaging are expensive, and the RL-style way of learning is clearly needed, while a big model is not. Against the usual stopping rules, the learned rule ties in normal cases and is much safer in bad ones.

Next, I will use realistic costs, compare against rules based on the closest papers, and, most importantly, let the system decide how many robots keep working instead of just when to stop. Two-level learning with MARL comes only if that shows a real gain.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021. pages 5 and 12

[2] Guillermo Amorín, Facundo Benavides, and Eduardo Grampín. A novel stop criterion to support efficient multi-robot mapping. In *2019 Latin American Robotics Symposium (LARS), 2019 Brazilian Symposium on Robotics (SBR) and 2019 Workshop on Robotics in Education (WRE)*, pages 369–374. IEEE, 2019. pages 3, 10, and 11

[3] Sebastian Becker, Patrick Cheridito, and Arnulf Jentzen. Deep optimal stopping. *Journal of Machine Learning Research*, 20, 2019. pages 11

[4] Wolfram Burgard, Mark Moors, Cyrill Stachniss, and Frank Schneider. Coordinated multi-robot exploration. *IEEE Transactions on Robotics*, 21(3), 2005. pages 1, 5, and 11

[5] Jimmy Chiun, Shizhe Zhang, Yizhuo Wang, Yuhong Cao, and Guillaume Sartoretti. MARVEL: Multi-agent reinforcement learning for constrained field-of-view multi-robot exploration in large-scale environments. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025. pages 11

[6] Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6, 2005. pages 4 and 11

[7] Dongge Han, Wendelin Böhmer, Michael Wooldridge, and Alex Rogers. Multi-agent hierarchical reinforcement learning with dynamic termination. In *Pacific Rim International Conference on Artificial Intelligence (PRICAI)*, 2019. pages 9 and 11

[8] Ninad Jadhav, Meghna Behari, Robert J. Wood, and Stephanie Gil. WiSER-X: Wireless signals-based efficient decentralized multi-robot exploration without explicit information exchange. *IEEE Robotics and Automation Letters*, 2026. pages 3 and 11

[9] Kim, Baek, Corah, Best, Moon, and Scherer. PRoID: predicted rate of information delivery in multi-robot exploration and relaying. arXiv preprint arXiv:2604.10433, 2026. pages 3, 10, and 11

[10] Francis A. Longstaff and Eduardo S. Schwartz. Valuing American options by simulation: a simple least-squares approach. *Review of Financial Studies*, 14(1), 2001. pages 5, 8, and 11

[11] Matteo Luperto, Marco Maria Ferrara, Matteo Princisgh, Giacomo Boracchi, and Francesco Amigoni. Estimating map completeness in robot exploration. *Autonomous Robots*, 50(1):6, 2026. pages 3, 10, and 11

[12] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016. pages 12

[13] Julio A. Placed and José A. Castellanos. Enough is enough: towards autonomous uncertainty-driven stopping criteria. *IFAC-PapersOnLine*, 55(14), 2022. pages 3 and 11

[14] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: a framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112, 1999. pages 9 and 11

[15] Hadar Szostak and Kobi Cohen. Deep multi-agent reinforcement learning for decentralized active hypothesis testing. *IEEE Access*, 12, 2024. pages 12

[16] Aaron Hao Tan, Federico Pizarro Bejarano, Yuhan Zhu, Richard Ren, and Goldie Nejat. Deep reinforcement learning for decentralized multi-robot exploration with macro actions. *IEEE Robotics and Automation Letters*, 8(1), 2023. pages 11

[17] John N. Tsitsiklis and Benjamin Van Roy. Optimal stopping of Markov processes: Hilbert space theory, approximation algorithms, and an application to pricing high-dimensional financial derivatives. *IEEE Transactions on Automatic Control*, 44(10), 1999. pages 5 and 11

[18] Brian Yamauchi. A frontier-based approach for autonomous exploration. In *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*, 1997. pages 1, 5, and 11

[19] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35:24611–24624, 2022. pages 9 and 11